

Algoritmos e Programação

Capítulo 2 — Fundamentos

Fonte: MEDINA, Marco. Algoritmos e Programação: Teoria e Prática. 2ª ed.
Seções: 2.1 a 2.4

2.1 — Manipulação de Dados

Uma das principais funcionalidades de um computador é a manipulação de informações. Apesar de internamente o computador manipular unicamente números, as linguagens de programação permitem que utilizemos outros tipos de informações nos programas de forma transparente.

2.1.1 — Tipos

Um **tipo de dado** é um conjunto de objetos que têm em comum o mesmo comportamento diante de um conjunto definido de operações.

Analogia: todas as pessoas formam um tipo — compartilham operações como comer, dormir, andar. Aviões formam outro tipo — voar, pousar, decolar. Porém, o conjunto {1, 'a', José da Silva, Fusca} não é um tipo, pois seus elementos não têm comportamento similar sob um mesmo conjunto de operações.

Tipos numéricos

- **Inteiros (Z)**: suportam adição, subtração, multiplicação etc.
- **Reais (R)**: idem, incluindo divisão com parte fracionária.

Outros tipos

- **Letras (caracteres)**: operações como escrever, ler, concatenar.
- **Lógicos/Booleanos**: apenas dois valores — Verdadeiro (V) e Falso (F).

■ Limitação das linguagens — As linguagens de programação não podem manipular todos os tipos do mundo real. A construção de algoritmos deve lidar exclusivamente com os tipos definidos na linguagem.

Valores lógicos (booleanos)

Originam-se da lógica de Boole. Qualquer afirmação (proposição) é Verdadeira ou Falsa:

Proposição	Valor
Cachorros são mamíferos	V
Cachorros são aves	F
Homens pensam	V
$5 > 3$	V
$7 = 3$	F

Operadores lógicos e tabelas-verdade

Operador **e** (conjunção):

e	v	f
v	v	f
f	f	f

Operador **ou** (disjunção):

ou	v	f
v	v	v
f	v	f

Operador **não** (negação): operador **unário**.

não	—
v	f
f	v

Precedência dos operadores lógicos

Precedência	Operador
1 (maior)	Parênteses
2	não
3	e, ou

Operadores aritméticos adicionais

Além de +, -, *, /, o livro define:

- **pot** — potência: $10 \text{ pot } 2 = 100$, $6 \text{ pot } 3 = 216$
- **raiz** — radiciação: $9 \text{ raiz } 2 = 3$, $216 \text{ raiz } 3 = 6$
- **div** — divisão inteira: $10 \text{ div } 4 = 2$, $22 \text{ div } 6 = 3$
- **resto** — resto da divisão inteira: $10 \text{ resto } 4 = 2$, $22 \text{ resto } 6 = 4$, $20 \text{ resto } 5 = 0$

Precedência dos operadores aritméticos

Precedência	Operadores
1 (maior)	Parênteses
2	pot, raiz
3	*, /, div, resto
4 (menor)	+, -

2.1.2 — Constantes e Variáveis

Tipos são conjuntos com comportamento comum, mas na prática precisamos manipular elementos individuais. Assim como identificamos um carro pela placa (ex.: AJF 5882), identificamos dados na memória por um nome (identificador).

Analogia do livro: variáveis e constantes são como garagens. O nome identifica a garagem, mas não garante o conteúdo. Se guardarmos um Fusca na garagem da Ferrari, ao buscar a Ferrari encontraremos

o Fusca.

■ Lição da analogia — Convém dar nomes significativos (mnemônicos) às variáveis, que lembrem o uso no algoritmo.

Diferença entre variável e constante

Conceito	Valor muda durante execução?	Quando é definido?
Constante	Não	No início do programa
Variável	Sim	Pode mudar a cada instrução

Declaração de variáveis

```
var <identificador1>[, <identificador2>, ...]: <tipo1>;  
    <identificador3>[, <identificador4>, ...]: <tipo2>;
```

Declaração de constantes

```
constante <identificador1> = <valor1>;  
    <identificador2> = <valor2>;
```

Regras para identificadores

- Compostos de **letras**, **números** e **_** (sublinhado).
- **Não** podem começar com número.
- Palavras em **negrito** são reservadas e não podem ser usadas como nomes.

Exemplo de declarações

```
constante MÁXIMO = 100;  
var quantidade, filhos, netos: inteiro;  
    aberto: booleano;  
    altura: real;  
    resposta: caractere;
```

Convenções da notação

- Palavras em **negrito** → palavras reservadas.
- Palavras entre **< >** → elementos a serem substituídos pelo programador.
- Elementos entre **[]** → opcionais.
- Textos entre **{ }** → comentários (ignorados na execução).

2.1.3 — Atribuição

A **atribuição** é o ato de colocar um valor dentro de uma variável. É representada pelo símbolo **←**.

```
<identificador da variável> ← <valor do mesmo tipo da variável>;  
<identificador da variável> ← <operações cujo resultado é do mesmo tipo>;
```

Exemplos

```

altura ← 1.80;
filhos ← 3;
netos ← filhos * 2 - 3;
quantidade ← MÁXIMO - 10;      { Consideramos MÁXIMO com valor 100 }
aberto ← F;
aberto ← aberto e V;           { aberto recebe F e V, que é F }
aberto ← quantidade < MÁXIMO;  { Como quantidade < MÁXIMO, aberto recebe F }
resposta ← 'S';

```

■ Compatibilidade de tipos — Apenas valores pertencentes ao tipo da variável podem lhe ser atribuídos. Operações lógicas seguem a álgebra de Boole; aritméticas seguem as regras de precedência.

2.2 — Blocos de Instruções

As instruções de uma linguagem de programação são executadas em sequência. Um bloco de instruções delimita onde essa sequência inicia e termina. Todo algoritmo tem pelo menos um bloco — o próprio algoritmo.

Estrutura de um bloco

```

<declaração de variáveis>
início
    <primeira instrução do bloco>
    <segunda instrução do bloco>
    ...
    <última instrução do bloco>
fim

```

Exemplo completo

```

constante MÁXIMO = 100;
var quantidade, filhos, netos: inteiro;
    aberto: booleano;
    altura: real;
início
    altura ← 1.80;
    filhos ← 3;
    netos ← filhos * 2 - 3;
    quantidade ← MÁXIMO - 10;
    aberto ← F;
    aberto ← aberto e V;
    aberto ← quantidade < MÁXIMO;
    resposta ← 'S';
fim

```

■ Isso não é um algoritmo — O código acima não realiza nenhuma tarefa útil — serve apenas como exemplo de formato. Um algoritmo deve resolver um problema.

2.2.1 — Problema 1: Vértices de um cubo

Enunciado: Sabendo que a relação entre vértices, arestas e faces é dada pela fórmula $\text{vértices} + \text{faces} = \text{arestas} + 2$, calcule o número de vértices de um cubo (6 faces e 12 arestas).

```

Algoritmo Vértices_Cubo

var vértices, faces, arestas: inteiro;
início
    faces ← 6;
    arestas ← 12;
    vértices ← arestas + 2 - faces;
fim

```

Resultado: $\text{vértices} = 12 + 2 - 6 = 8 \checkmark$

2.2.2 — Problema 2: Raízes de equação de 2º grau

A equação $ax^2 + bx + c = 0$ tem raízes reais pela fórmula de Bhaskara:

$$x' = (-b + \sqrt{\Delta}) / 2a, \quad x'' = (-b - \sqrt{\Delta}) / 2a, \quad \Delta = b^2 - 4ac$$

Encontre as raízes de $2x^2 + 4x - 3 = 0$.

```

Algoritmo Raízes

var delta, x1, x2, a, b, c: real;
início
    a ← 2;
    b ← 4;
    c ← -3;
    delta ← b pot 2 - 4 * a * c;
    x1 ← (-b + (delta raiz 2)) / 2 * a;
    x2 ← (-b - (delta raiz 2)) / 2 * a;
fim

```

2.3 — Entrada e Saída

Um programa pode ser visto de forma sistêmica: entrada → processamento → saída.

Entrada → [Processamento] → Saída

- **Entrada:** teclado, mouse, scanner etc. (padrão: teclado).
- **Saída:** vídeo, impressora etc. (padrão: vídeo/monitor).

Papéis do desenvolvedor e do usuário

Desenvolvedor	Usuário
Desenvolve o algoritmo (pseudocódigo)	Utiliza o programa executável
Transforma em programa (linguagem)	Fornece dados de entrada
Gera código executável (compilação)	Analisa os resultados (saída)

■ Por que entrada e saída são necessárias? — Os algoritmos da seção 2.2 resolvem sempre o mesmo problema com os mesmos valores fixos. Com entrada e saída, o programa se torna adaptável — o usuário informa os dados e recebe resultados sem precisar alterar o código-fonte.

2.3.1 — Entrada

A entrada é o meio pelo qual o usuário informa dados ao programa. É feita pelo comando:

```
leia(<variável>);
```

A leitura implica o uso de uma variável, pois o valor precisa ser armazenado para futura manipulação.

2.3.2 — Saída

A saída é feita pelo comando:

```
escreva(<valor>);
```

O valor pode ser um texto (entre aspas), o conteúdo de uma variável/constante, ou uma combinação intercalando textos e variáveis separados por vírgulas.

```
escreva("O resultado é: ", variável1);  
escreva("O valor ", variável1, " é o resultado final.");
```

2.3.2.1 — Problema 3: Vértices de objeto genérico

Enunciado: Calcule o número de vértices de um objeto geométrico genérico. A entrada será o número de faces e arestas; a saída será o número de vértices.

```
Algoritmo Vértices_Objeto_Geométrico  
  
var vértices, faces, arestas: inteiro;  
início  
    escreva("Entre com o número de faces: ");  
    leia(faces);  
    escreva("Entre com o número de arestas: ");  
    leia(arestas);  
    vértices ← arestas + 2 - faces;  
    escreva("O número de vértices é: ", vértices);  
fim
```

■ Diferença do Problema 1 — No Problema 1, faces e arestas eram fixos no código (cubo). Agora o algoritmo funciona para qualquer objeto geométrico — o usuário fornece os dados via leia.

2.4 — Teste de Mesa

É difícil verificar se um algoritmo está correto apenas lendo o código. A correção de um algoritmo é essencial: ele só é correto se produz o resultado esperado para qualquer entrada possível.

O teste de mesa é a simulação manual da execução, passo a passo, instrução a instrução, verificando o estado de todas as variáveis a cada linha.

Como fazer

1. Numerar as linhas que contêm instruções.
2. Criar uma tabela com uma coluna para cada variável.
3. Percorrer linha a linha, atualizando os valores.

Convenções:

- Valores de **entrada** (lidos) ficam entre colchetes [].
- Valores de **saída** (escritos) ficam entre chaves { }.
- ? indica valor indefinido/lixo.

Exemplo — Cubo (faces=6, arestas=12)

Linha	vértices	faces	arestas	Comentário
1	?	?	?	Valores indefinidos
2	?	[6]	?	faces lida
3	?	6	?	
4	?	6	[12]	arestas lida
5	8	6	12	Cálculo de vértices
6	{8}	6	12	Saída

Exemplo — Pirâmide triangular (faces=4, arestas=6)

Linha	vértices	faces	arestas
1	?	?	?
2	?	[4]	?
3	?	4	?
4	?	4	[6]
5	4	4	6
6	{4}	4	6

2.4.1 — Problema 4: Intercambiar valores

Enunciado: Faça um algoritmo que armazene dois números em duas variáveis e que troque os valores das variáveis.

■ Dica do livro — Cuidado com a sobreposição de valores nas variáveis.

Versão 1 — Tentativa ingênua (INCORRETA)

```
Algoritmo Troca_Valores_Versão1  
  
var a, b: inteiro;  
início  
1■ leia(a, b);  
2■ a ← b;  
3■ b ← a;  
4■ escreva(a, b);  
fim
```

Teste de mesa (entrada: a=4, b=7):

Linha	a	b
1	[4]	[7]
2	7	7
3	7	7

Linha	a	b
4	{7}	{7}

Resultado: não houve troca! O valor 4 (de a) foi perdido pela sobreposição na linha 2.

■ Por que falha? — Cada variável mantém apenas um valor por vez. A atribuição $a \leftarrow b$ destruiu o valor original de a para sempre.

Versão 2 — Com variável auxiliar (CORRETA)

Assim como duas pessoas não podem trocar de lugar sem um espaço intermediário, precisamos de uma variável auxiliar (aux).

Algoritmo Troca_Valores_Versão2

```
var a, b, aux: inteiro;
início
1■ leia(a, b);
2■ aux ← a;
3■ a ← b;
4■ b ← aux;
5■ escreva(a, b);
fim
```

Teste de mesa (entrada: a=4, b=7):

Linha	a	b	aux
1	[4]	[7]	?
2	4	7	4
3	7	7	4
4	7	4	4
5	{7}	{4}	4

Resultado: troca realizada com sucesso! ✓

■ Lição principal — O teste de mesa revelou o erro da Versão 1 que não era óbvio apenas lendo o código. Sempre faça teste de mesa para validar a lógica dos seus algoritmos.

Resumo Rápido

Fundamentos (Medina - Cap. 2)

■■■ 2.1 Manipulação de Dados

■ ■■■ 2.1.1 Tipos

- ■ ■■■ Numéricos: inteiro, real
- ■ ■■■ Caractere
- ■ ■■■ Lógico/Booleano (V, F)
- ■ ■■■ Operadores lógicos: e, ou, não (tabelas-verdade)
- ■ ■■■ Operadores aritméticos: +, -, *, /, pot, raiz, div, resto

■ ■■■ 2.1.2 Constantes e Variáveis

- ■ ■■■ Constante: valor fixo, definido uma vez
- ■ ■■■ Variável: valor mutável durante execução
- ■ ■■■ Identificadores (nomes significativos)
- ■ ■■■ Declaração: var/constante

■ ■■■ 2.1.3 Atribuição

- ■■■ variável ← expressão (compatível em tipo)

■

■■■ 2.2 Blocos de Instruções

- ■■■ início ... fim (execução sequencial)

■ ■■■ 2.2.1 Problema 1: Vértices do cubo ($V = A + 2 - F$)

■ ■■■ 2.2.2 Problema 2: Raízes eq. 2º grau (Bhaskara)

■

■■■ 2.3 Entrada e Saída

- ■■■ Entrada → Processamento → Saída

■ ■■■ 2.3.1 Entrada: leia(variável)

■ ■■■ 2.3.2 Saída: escreva(valor)

■ ■■■ 2.3.2.1 Problema 3: Vértices genérico (com E/S)

■

■■■ 2.4 Teste de Mesa

- ■■■ Simulação passo a passo com tabela

- ■■■ [] = entrada, { } = saída, ? = indefinido

■ ■■■ 2.4.1 Problema 4: Troca de variáveis

- ■■■ Versão 1: sem aux → FALHA (sobreposição)

- ■■■ Versão 2: com aux → CORRETO

#algoritmos #programação #medina #fundamentos #pseudocódigo #teste-de-mesa #tipos #variáveis #atribuição #entrada-saída